

Car Marketplace System

Object Oriented Programming — Assignment 02

Girdhari Lal

FAST NUCES Karachi | Spring 2026

1. Introduction

This is Assignment 02 which extends Assignment 01. We added Inheritance, Polymorphism, Abstraction, Operator Overloading, and Friend Functions to the existing Car Marketplace system. All new concepts are built on top of the original 10 classes.

2. Inheritance (08 Marks)

We implemented 5 inheritance relationships. The main idea is that Seller, Buyer, and Admin all share common properties like id, name, and phone, so we made a base class User. Similarly, Car and Bike both share brand, model, year, and mileage, so we made a base class Vehicle.

Base Class	Derived Class	Why
Vehicle	Car	Car is a Vehicle with fuelType added
Vehicle	Bike	Bike is a Vehicle with bikeType added
User	Seller	Seller is a User who posts listings
User	Buyer	Buyer is a User who searches and buys
User	Admin	Admin is a User who manages the platform

The User base class holds id, name, phone, and isActive which are common to all user types. Vehicle holds brand, model, year, mileage, and color which are common to all vehicle types. This avoids repeating the same code in every class.

3. Polymorphism (05 Marks)

We used both function overloading and function overriding. The display() function and showReport() function are overridden in each subclass so the same function call gives different output depending on the object type.

- Car::display() prints [Car] with fuel type. Bike::display() prints [Bike] with bike type.
- Seller::showReport() prints seller rating and listing count.
- Buyer::showReport() prints buyer budget and favorites count.
- Admin::showReport() prints how many listings were approved and removed.

In main() we show runtime polymorphism using Vehicle pointers:

```
Vehicle* v1 = &car; v1->display(); // prints Car details
```

```
Vehicle* v2 = &bike; v2->display(); // prints Bike details
```

Same pointer type, different output. This is polymorphism.

4. Abstraction (10 Marks)

We created 4 abstract classes with pure virtual functions. These classes define interfaces that other classes must follow. No object of these classes can be created directly.

Abstract Class	Pure Virtual Functions	Who Implements It
IDisplayable	display(), getType()	Listing, Car, Bike
ISearchable	matchesKeyword(), inBudget()	Listing, Car, Bike
IReportable	showReport(), getID()	Seller, Buyer, Admin
Vehicle	display(), getType(), matchesKeyword()	Car, Bike

The assignment also asked for separate .h and .cpp files for abstract classes. Since this is a single-file submission, the abstract classes are defined at the top of the file and all concrete classes implement their pure virtual functions below them.

5. Operator Overloading (07 Marks)

We overloaded 4 operators across different classes. Each one has a clear reason for why it is needed.

Class	Operator	Compares By	Why Needed
Vehicle	==	brand + model	Check if two listings have the same car
Listing	==	listing ID	Check if buyer already saved this listing
Listing	<	price	Find the cheaper option between two listings
User	==	user ID	Check if a user already exists in the system

6. Friend Functions (05 Marks)

We used 3 friend functions. A friend function can access private members of a class without being a member of that class. We used them when a function needs private data from two different objects at the same time.

Function	Accesses Private	Why Friend
printCheaper(Listing&, Listing&)	price	Needs price from both Listing objects directly
compareSellerRatings(Seller&, Seller&)	rating, name	Needs rating from both Sellers at once
canAfford(Buyer&, Listing&)	budget	Checks if buyer budget covers listing price

7. Integration with Assignment 01 (05 Marks)

All features from Assignment 01 still work. The new classes extend the old ones without breaking anything:

- Seller and Buyer now extend User but all their original functions like saveFavorite(), addListingID(), and showFavorites() still work the same way.
- Car now extends Vehicle but all Listing functions that use Car still work because Car still has all the same getters.
- Admin still approves and removes listings. Marketplace still manages all arrays.
- All 10 original classes are still present. We added 4 abstract classes on top.

8. Conclusion

Assignment 02 extends the Car Marketplace system with all five required OOP concepts. Inheritance removes duplicate code by putting shared data in User and Vehicle base classes. Polymorphism lets the same function call behave differently for Seller, Buyer, Admin, Car, and Bike. Abstraction defines clear interfaces through 4 abstract classes. Operator overloading makes object comparisons natural. Friend functions allow cross-object access to private data when needed. The program compiles and runs without errors.